DEVMarket

PROJECT DOCUMENTATION

A Web-Based Digital Project Marketplace

Student Name: Kavya Ishwar Medar

AF Code: AF05331477

Frontend: React.js
Backend: PHP with Laravel Framework
Database: MySQL

Prepared for academic and project-development use

Document Information

Item	Details
Project Title	DevMarket – Digital Project Marketplace
Project Type	Web-based marketplace application
Frontend	React.js, HTML5, CSS3, JavaScript
Backend	PHP using Laravel Framework; RESTful API
Database	MySQL
Primary User Roles	Admin, Seller, Buyer
Document Version	1.0 – Draft for review

Note: The Google Docs link supplied for the reference format could not be read from this chat. This document is therefore prepared as a complete standard project report using the DevMarket requirements already discussed. If the reference document has a specific college template or section order, it can be aligned after the document is uploaded or its contents are pasted here.

Abstract

DevMarket is a web-based digital marketplace designed to connect people who develop software projects with people who want to purchase them. The platform provides three main roles: Admin, Seller, and Buyer. Sellers can create project listings containing a title, description, category, price, screenshots, and downloadable files. Each listing is submitted for review, and only projects approved by the Admin are published in the marketplace. Buyers can browse and search approved projects, view project details, purchase projects, access their order history, download purchased files, and communicate with sellers through an integrated chat feature. The frontend is developed using React.js, the backend uses PHP with the Laravel framework to expose RESTful APIs, and MySQL stores application data. The system aims to improve project discovery, listing review, communication, and transaction record management in one centralized application.

Table of Contents

- 1. Introduction
- 2. Problem Definition
- 3. Objectives
- 4. Existing System
- 5. Proposed System
- 6. Scope of the Project
- 7. Project Modules
- 8. System Requirements
- 9. Technology Stack
- 10. System Architecture
- 11. Database Design
- 12. Functional Requirements
- 13. Non-Functional Requirements
- 14. Main Workflows

- 15. Security and Validation
- 16. Testing Plan
- 17. Limitations
- 18. Future Enhancements
- 19. Conclusion
- 20. References

1. Introduction

Software developers and students often create reusable projects, applications, templates, and source-code packages that may be useful to other people. However, discovering suitable projects, evaluating their features, contacting the developer, and managing purchases may require visiting several websites or communicating through separate channels.

DevMarket proposes a single online platform where sellers can publish digital project listings and buyers can discover and purchase approved projects. An administrator supervises project submissions to maintain listing quality and platform rules. The application separates the user interface from the backend: React.js provides the interactive frontend, Laravel handles business logic and API requests, and MySQL stores users, projects, orders, messages, and related records.

2. Problem Definition

In an informal project-selling process, project details may be incomplete, listing quality may be inconsistent, and buyers may find it difficult to compare projects or verify the information provided. Sellers may not have a structured way to manage submissions and sales, while buyers and sellers may need to use separate communication tools. Without centralized records, project approval, order tracking, and access to purchased files can become difficult to manage.

DevMarket addresses these problems by providing role-based access, a moderated project catalogue, structured project details, purchase and order records, controlled downloads, and buyer-seller messaging in one system.

3. Objectives

1. Develop a responsive digital marketplace for software and academic/portfolio projects.
2. Provide separate access and dashboards for Admin, Seller, and Buyer roles.
3. Allow sellers to submit project listings and monitor approval status.
4. Allow the Admin to review, approve, or reject submitted projects.
5. Allow buyers to search, filter, view, and purchase approved projects.
6. Maintain order and payment-status records and provide access to purchased files.
7. Provide a chat facility for buyer-seller communication.
8. Protect user accounts, project files, and role-restricted actions.

4. Existing System

In a basic or manual approach, developers may share projects through social media, messaging applications, personal websites, or general file-sharing platforms. These methods may not provide a consistent project

catalogue, dedicated approval workflow, integrated purchase history, or a single place for buyer-seller communication.

4.1 Disadvantages of the Existing Approach

- Project information can be scattered across multiple platforms.
- Buyers may find it difficult to compare project descriptions, categories, and prices.
- Listings may be published without a review process.
- Purchase and delivery records may be maintained manually.
- Buyer-seller communication and project files may be stored separately.

5. Proposed System

The proposed DevMarket system centralizes project listing, moderation, discovery, purchase records, file access, and messaging. Sellers submit projects through a form. The backend validates the request and stores the project with a pending status. The Admin reviews the submission and approves or rejects it. Approved listings become visible to buyers. After a successful purchase record is created, the buyer can view the order and access the associated project download according to the platform's access rules.

5.1 Advantages

- Centralized catalogue of approved digital projects.
- Clear project approval status for sellers.
- Structured search and category-based browsing for buyers.
- Role-based dashboards and permissions.
- Order history and payment-status tracking.
- Project-related buyer-seller communication.
- Improved organization of listings and digital delivery.

6. Scope of the Project

6.1 Included Scope

- Account registration, login, logout, and role-based access.
- Admin management of users, categories, and project submissions.
- Seller profile and project listing management.
- Project title, description, category, price, screenshots, and file metadata.
- Search, filtering, sorting, and project detail pages.
- Order creation and payment status records.
- Download access for eligible completed orders.
- Buyer-seller messaging and message history.
- Basic reviews and ratings for eligible buyers, if included in the implemented version.

6.2 Out of Scope for the Initial Version

- Freelance service bidding or job-contract management.
- Automatic code-quality analysis of every uploaded project.
- Guaranteed virus-free or defect-free third-party source code.
- Multi-currency tax handling and complex international payout operations.

- Native Android or iOS applications.

7. Project Modules

7.1 Authentication and Authorization

Registration, login, logout, password reset, profile management, and role-based permissions for Admin, Seller, and Buyer.

7.2 Admin Module

Admin dashboard; view and manage users; manage categories; review submitted projects; approve or reject listings with a reason; monitor orders and reported content.

7.3 Seller Module

Seller profile; create, edit, and manage project listings; upload screenshots and project packages; submit listings for review; view approval status and sales/order summaries.

7.4 Buyer Module

Browse approved projects; search and filter listings; view project descriptions and seller information; purchase projects; view order history; download eligible purchased files; communicate with sellers.

7.5 Project Catalogue and Review

Store project details and moderation status. Only approved and active projects should be displayed in the public catalogue.

7.6 Order Module

Create an order for a selected project, store the buyer, project, price snapshot, order status, and timestamps, and show order history.

7.7 Payment Module

Record payment reference, amount, provider, and status. In a demonstration version, payment may be simulated; real payments require a payment gateway and verified server-side callbacks.

7.8 Chat Module

Allow a buyer to start or continue a conversation with a seller, send messages, and view conversation history. Access must be restricted to conversation participants and authorized administrators where applicable.

7.9 Reviews and Ratings

Allow eligible buyers to submit a rating and review for a purchased project, and display valid reviews on project pages.

7.10 Notifications

Show in-app notifications for project approval or rejection, order updates, new messages, and other important account events.

8. System Requirements

8.1 Hardware Requirements

Component	Minimum Suggested Requirement
Processor	Dual-core processor or better
RAM	4 GB minimum; 8 GB recommended for development
Storage	At least 5 GB free space for source code, dependencies, and test files
Internet	Required for package installation, online resources, and deployment

8.2 Software Requirements

Software	Purpose
Windows, Linux, or macOS	Development operating system
PHP version supported by the selected Laravel release	Backend language runtime
Composer	PHP dependency management
Laravel	Backend framework and REST API
Node.js and npm	React development and frontend dependencies
React.js	Frontend user interface
MySQL	Relational database
VS Code or another IDE	Code editing and debugging
Postman or equivalent	API testing
Git	Version control
Modern web browser	Application use and testing

9. Technology Stack

Layer	Technology	Role in DevMarket
Frontend	React.js	Reusable components, pages, forms, state, and user interactions
Markup and styling	HTML5, CSS3, JavaScript	Structure, responsive styling, and browser-side behavior
Backend	PHP with Laravel	Business rules, validation, authentication, authorization, and REST APIs
API communication	JSON over HTTP/HTTPS	Data exchange between React and Laravel
Database	MySQL	Persistent storage for accounts, listings, orders, messages, and records
File storage	Laravel storage / configured object	Project packages and listing images; private files should not be publicly

	storage	exposed
Development tools	Composer, npm, Git, Postman	Dependencies, builds, source control, and API testing

Implementation note: React and Laravel are separate application layers. React sends HTTP requests to Laravel API endpoints. Laravel validates requests, checks permissions, reads or writes MySQL data through Laravel's database layer, and returns JSON responses.

10. System Architecture

10.1 High-Level Architecture

Layer	Responsibilities
Presentation Layer – React.js	Pages, navigation, forms, search results, dashboards, and client-side validation.
API Layer – Laravel	Routes, controllers, request validation, authentication, authorization, and consistent JSON responses.
Business Logic Layer – Laravel services/models	Project moderation, order rules, download eligibility, chat permissions, and related application rules.
Data Layer – MySQL	Users, projects, categories, orders, payments, conversations, messages, reviews, and notifications.
File Storage	Project packages and images; private downloads are authorized by the backend.

10.2 Request Flow

9. The user performs an action in the React interface.
10. React sends a request to the relevant Laravel API endpoint.
11. Laravel authenticates the user and validates the request.
12. Authorization and business rules are checked.
13. Laravel reads or updates MySQL and manages file access where required.
14. Laravel returns a JSON response and an appropriate HTTP status code.
15. React updates the interface based on the response.

11. Database Design

The following is a proposed logical database design. Exact columns and relationships should be finalized during implementation and reflected in Laravel migrations.

Table	Proposed Fields	Purpose
users	id (PK), name, email (unique), password, role, status, email_verified_at, timestamps	Stores Admin, Seller, and Buyer accounts.
categories	id (PK), name, slug (unique), description, status, timestamps	Groups projects for browsing.
projects	id (PK), seller_id (FK), category_id	Stores project listings and moderation

	(FK), title, slug, description, price, status, rejection_reason, thumbnail_path, package_path, timestamps	status.
project_images	id (PK), project_id (FK), image_path, alt_text, timestamps	Stores listing screenshots or images.
orders	id (PK), order_number (unique), buyer_id (FK), project_id (FK), amount, status, purchased_at, timestamps	Stores buyer purchases and order state.
payments	id (PK), order_id (FK), provider, provider_reference, amount, status, paid_at, timestamps	Stores payment transaction metadata and status.
conversations	id (PK), buyer_id (FK), seller_id (FK), project_id (FK, nullable), timestamps	Stores buyer-seller conversation context.
messages	id (PK), conversation_id (FK), sender_id (FK), message_body, read_at, timestamps	Stores messages within conversations.
reviews	id (PK), buyer_id (FK), project_id (FK), order_id (FK), rating, comment, status, timestamps	Stores buyer reviews; enforce purchase eligibility.
notifications	id (PK), user_id (FK), title, body, type, read_at, timestamps	Stores in-app notifications.

11.1 Main Relationships

- One seller (users record with Seller role) can own many projects.
- One category can contain many projects.
- One buyer can place many orders; each order refers to a project.
- An order may have one or more payment attempts, depending on implementation.
- A conversation links a buyer and a seller and may relate to a project.
- A conversation contains many messages; each message has one sender.
- A project can receive multiple reviews, subject to purchase and review rules.
- A user can receive many notifications.

11.2 Suggested Project Status Values

Status	Meaning
draft	Seller is still preparing the listing.
pending	Submitted and waiting for Admin review.
approved	Accepted and eligible for publication.
rejected	Not accepted; the seller can see the reason.
inactive	Temporarily hidden or disabled.

Use database foreign keys, indexes, unique constraints, and transactions where appropriate. Store passwords only as secure hashes using Laravel's authentication facilities; never store plaintext passwords.

12. Functional Requirements

ID	Requirement
FR-01	The system shall allow users to register and log in securely.
FR-02	The system shall enforce role-based access for Admin, Seller, and Buyer.
FR-03	A Seller shall be able to create and submit a project listing.
FR-04	The system shall set submitted projects to pending until reviewed.
FR-05	The Admin shall be able to approve or reject submitted projects.
FR-06	Only approved and active projects shall appear in the public catalogue.
FR-07	A Buyer shall be able to search, filter, and view project details.
FR-08	The system shall create an order record for a purchase attempt and update it according to payment outcome.
FR-09	Only eligible buyers shall be allowed to download purchased project packages.
FR-10	Buyers and sellers shall be able to exchange messages through authorized conversations.
FR-11	The system shall preserve order, payment, and message history.
FR-12	The system shall validate inputs and provide clear success or error messages.

13. Non-Functional Requirements

- Security: protect credentials, validate requests, authorize sensitive actions, and restrict access to private downloads.
- Usability: provide clear navigation and role-specific dashboards.
- Performance: paginate project listings and avoid loading large project files into listing pages.
- Reliability: use database constraints and transactions for important order/payment updates.
- Maintainability: separate React components, Laravel controllers, services, models, and request validation.
- Compatibility: support current mainstream desktop and mobile browsers.
- Scalability: allow the API and file-storage layers to be deployed independently as the application grows.
- Accessibility: use readable text, labels, keyboard-friendly controls, and meaningful validation messages.

14. Main Workflows

14.1 Seller Project Submission

16. Seller logs in and opens the seller dashboard.
17. Seller completes the project form and uploads permitted images and a project package.

18. React sends the form data to Laravel.
19. Laravel validates required fields, file type, file size, price, and ownership.
20. The project is saved with pending status.
21. Admin reviews the submission and approves or rejects it.
22. The seller can view the final status and any rejection reason.

14.2 Buyer Purchase

23. Buyer logs in and opens an approved project page.
24. Buyer selects the purchase action.
25. Laravel checks project availability and creates the order in a pending state.
26. If a payment gateway is integrated, the buyer completes payment through the provider.
27. The backend verifies the payment result server-side before marking the order as paid/completed.
28. The buyer sees the order in purchase history and receives authorized download access.

For a student demonstration, a clearly labelled simulated-payment mode may be used. It must not be presented as a real payment integration.

14.3 Buyer-Seller Chat

29. An authenticated buyer opens a project and starts a conversation with its seller.
30. Laravel verifies that the user is allowed to access the conversation.
31. The message is saved with its sender and timestamp.
32. The recipient sees the message in the conversation.
33. Only authorized participants can read or send messages in that conversation.

15. Security and Validation

- Use Laravel authentication and authorization middleware or policies for protected endpoints.
- Validate and normalize all server-side input; client-side validation is only a usability aid.
- Use password hashing and never log passwords or sensitive payment details.
- Use CSRF protection where cookie-based authentication requires it; configure CORS narrowly for the React origin.
- Apply rate limits to login, registration, messaging, and other abuse-prone endpoints.
- Validate uploaded file MIME type, extension, size, and storage location; generate safe filenames.
- Keep project packages in private storage and serve them through an authorization-checked download endpoint.
- Do not trust payment success reported only by the browser; verify provider callbacks/signatures on the server.
- Use HTTPS in production and store secrets in environment configuration rather than source code.
- Use database transactions and idempotent payment handling to reduce duplicate order/payment updates.

16. Testing Plan

Test ID	Test Scenario	Expected Result	Status
TC-01	Register with valid details	Account is created and the user can sign in	Planned
TC-02	Register with an already-	Validation error is shown; duplicate account is not	Planned

	used email	created	
TC-03	Log in with incorrect credentials	Access is denied with a safe error message	Planned
TC-04	Seller submits a valid project	Project is saved with pending status	Planned
TC-05	Buyer attempts to view a pending project in catalogue	Pending project is not publicly listed	Planned
TC-06	Admin approves a project	Project becomes visible if active	Planned
TC-07	Admin rejects a project	Project remains unpublished and rejection reason is stored	Planned
TC-08	Buyer tries to download without an eligible order	Download is denied	Planned
TC-09	Eligible buyer downloads purchased project	Authorized download is delivered	Planned
TC-10	User tries to access another conversation	API denies access	Planned
TC-11	Upload disallowed or oversized file	Server rejects the upload	Planned
TC-12	Payment provider reports failed payment	Order is not marked paid/completed	Planned

Testing status is marked Planned because the test results must be filled in after the application is implemented and executed.

17. Limitations

- The quality and correctness of uploaded third-party projects cannot be guaranteed solely by listing moderation.
- Real payment processing requires a supported gateway, merchant configuration, server-side verification, and testing.
- Real-time chat may require polling or a WebSocket service; the implementation approach affects responsiveness.
- Large project packages require file-size limits, sufficient storage, and reliable download handling.
- Seller identity verification, refunds, disputes, and tax compliance may require additional policies and integrations.

18. Future Enhancements

- Integrate a production payment gateway and seller payout workflow.
- Add email verification and email notifications.
- Add real-time messaging and online status.
- Add advanced filters for programming language, framework, project type, and compatibility.
- Add verified-purchase badges and review moderation.

- Add reporting dashboards for project views, sales, and platform activity.
- Add a project report/flag system and dispute-resolution workflow.
- Use cloud object storage and a content-delivery network for larger downloads.
- Add automated security scanning and clear user warnings for downloadable source code.
- Add deployment automation, backups, monitoring, and audit logs.

19. Conclusion

DevMarket is proposed as a centralized digital project marketplace that connects project sellers with buyers through a moderated and structured platform. The Admin supervises listings, Sellers manage project submissions, and Buyers discover and purchase approved projects. Features such as search, order tracking, authorized downloads, and messaging are intended to make the process more organized and convenient. React.js provides the frontend, PHP with Laravel implements the backend and RESTful APIs, and MySQL stores the application data. The system can be developed incrementally, beginning with authentication and project approval before adding purchases, payment integration, messaging, and optional reviews.

20. References

- Laravel Documentation – https://laravel.com/docs
- React Documentation – https://react.dev
- MySQL Documentation – https://dev.mysql.com/doc/
- PHP Manual – https://www.php.net/manual/en/
- Composer Documentation – https://getcomposer.org/doc/
- Node.js Documentation – https://nodejs.org/docs/

These references are official technical documentation sources for the proposed stack. Confirm the exact Laravel, PHP, React, and MySQL versions selected for implementation before final submission.